MINI-PROJECT CONCEPT

ImageSense AI

Intelligent Image Caption Generator
Using CNN and RNN (LSTM)

CORE IDEA
Upload an image → CNN extracts visual features → RNN (LSTM) generates a natural-language caption.

Purpose

A practical deep-learning mini-project that demonstrates how Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) can work together in one application.

Proposed implementation

Python • TensorFlow/Keras • CNN feature extractor • LSTM decoder • Streamlit interface

Prepared as a project discussion document for the project team.

1. Project Overview

ImageSense AI is an image-captioning application. The user uploads an image, and the system produces a short sentence describing its visible content. For example, an image of a dog running on grass may produce: *"A dog is running through a grassy field."*

Problem statement

Understanding an image requires identifying its visual contents and expressing that information in language. This project combines image feature extraction and sequence generation to produce an automatic text description.

Objectives

• Build a CNN–RNN pipeline rather than two unrelated demonstrations.
• Extract meaningful visual features from input images.
• Generate captions using an LSTM-based sequence decoder.
• Provide a simple interface for uploading an image and viewing its generated caption.
• Evaluate caption quality using suitable test examples and, where feasible, an automatic metric.

2. How CNN and RNN Work Together

The two neural-network components have different responsibilities but form one end-to-end image-captioning workflow.

STAGE	COMPONENT	ROLE
1	**Input image**	User uploads an image.
2	**CNN – MobileNetV2**	Extracts visual features such as patterns, shapes and object-related information. A pretrained CNN can be used as a feature extractor.
3	**Feature vector**	Represents the image in numerical form and is passed to the caption decoder.
4	**RNN – LSTM**	Uses the image features and previously generated words to predict the next word in the caption.
5	**Output caption**	Displays the completed sentence to the user.

3. Proposed Model Workflow

Image → Preprocessing → CNN feature extraction → Feature vector → LSTM caption decoder → Word-by-word prediction → Final caption

During training, the model learns from image–caption pairs. The CNN provides image information, while the LSTM learns the order and context of words. During prediction, the model starts with a beginning-of-sequence token and predicts words until an end token or a maximum caption length is reached.

4. Technology Stack

TOOL / TECHNOLOGY	USE IN THE PROJECT
Python	Main programming language.
TensorFlow / Keras	Build, train and run the CNN–LSTM model.

TOOL / TECHNOLOGY	USE IN THE PROJECT
MobileNetV2 (CNN)	Pretrained image feature extractor; its classification head can be removed for feature extraction.
LSTM (RNN)	Generate the caption as a sequence of words.
NumPy / Pandas	Numerical operations and dataset handling.
Streamlit	Create a simple image-upload web interface.
Matplotlib	Plot training loss and evaluation visuals if needed.

5. Dataset

Suggested dataset: Flickr8k

Flickr8k is a commonly used educational image-captioning dataset containing 8,000 images, with five human-written captions associated with each image. It provides paired visual and text data suitable for training and testing a small image-captioning model.

Important: Check the dataset's current access terms and download instructions before use. Keep training, validation and test images separated to avoid evaluating on images used for training. A small dataset and an LSTM model may produce imperfect or repetitive captions; that is normal for a learning-scale project.

6. Main Implementation Steps

1. Obtain the image files and their corresponding captions.
2. Clean and tokenize the captions; add start and end tokens.
3. Resize and preprocess images in the format expected by the selected CNN.
4. Use MobileNetV2 to extract image feature vectors.
5. Prepare caption input sequences and their next-word target labels.
6. Build an LSTM decoder that combines image features with caption sequence information.
7. Train the decoder on the training split and monitor validation performance.
8. Generate captions for images not used during training.
9. Connect the trained model to a Streamlit application.

7. Expected Application Features

• Upload an image (JPG / JPEG / PNG).
• Preview the uploaded image.
• Generate and display one caption.
• Show a clear message if the file is unsupported or processing fails.
• Optionally display a few test examples and training/evaluation results.

8. Evaluation and Results to Record

Test the application on images that were not used for training. Record the input image, generated caption and a brief observation about whether the caption correctly describes the main objects and action.

EVALUATION ITEM	WHAT TO REPORT
Training / validation loss	Plot the loss curves and note whether training is stable.
Example predictions	Show several test images with the captions generated by the model.
Caption metric (optional)	BLEU can be calculated against reference captions. Explain that automatic metrics do not fully measure caption meaning.
Limitations	Mention incorrect object names, incomplete sentences or repetitive captions if observed.

9. Suggested Project Scope

Recommended scope: Build a working prototype using a pretrained CNN as a fixed feature extractor and train an LSTM caption decoder on the selected dataset. This keeps the project focused on the CNN + RNN combination while avoiding the cost of training a large vision model from scratch.

The CNN and LSTM should be visibly connected in the implementation. Simply running an image classifier and a separate text-generation model would not demonstrate the intended combined architecture.

10. Initial Reference Resources

• Flickr8k dataset: search for the Flickr8k image-caption dataset and verify its access terms from the hosting source.
• TensorFlow / Keras documentation: https://www.tensorflow.org/ and https://keras.io/
• MobileNetV2 documentation: https://keras.io/api/applications/mobilenet/
• Streamlit documentation: https://docs.streamlit.io/

Decision for the project team
Confirm the project title, agree on the Flickr8k dataset, and decide whether to use Streamlit for the demo. This document is an initial project proposal; implementation, results, PPT and report can be prepared after the team agrees on the scope.